

Aurora PR20/PR21 Architectural Guidance

First-Class Presets & Palettes and Orchestration-Focused Song Mode

Purpose

As PR20 (Groups, Presets, Palettes) and PR21 (Song Mode) are implemented, preserve Aurora's existing architectural philosophy of strong domain models, stable references, clear separation of concerns, and deterministic engine behavior.

These features should not be implemented merely as UI conveniences. They are fundamental parts of Aurora's show model and should be designed accordingly.

1. PR20: Presets and Palettes Must Be First-Class Model Objects

Presets and palettes should exist as persistent entities within `AuroraModel`, with stable UUID-based identities.

Do **not** implement palettes simply as buttons that copy literal attribute values into the programmer or cue.

A palette should be something other Aurora entities can intentionally reference.

Conceptually:

```
struct Palette: Identifiable, Codable, Hashable {  
    let id: UUID  
    var name: String  
    var type: PaletteType  
    var values: PaletteValueData  
}
```

The exact implementation should follow existing Aurora conventions rather than necessarily using this literal structure.

Possible palette types include:

- Intensity
- Color

- Position
- Beam
- Gobo
- General/All

The architecture should remain extensible for additional types.

2. Referenced Values vs. Literal Values

Aurora should distinguish between:

1. A literal attribute value stored directly in a cue.
2. A reference to a palette/preset.

For example, a cue could explicitly contain:

```
Fixture Group: Back Wash  
Color: RGB(255, 120, 30)
```

or intentionally reference:

```
Fixture Group: Back Wash  
Color Palette: "Warm Amber"  
Palette ID: <UUID>
```

Those two cases should have different semantics.

The first is a fixed value.

The second says:

Use whatever "Warm Amber" currently means.

This distinction should survive project serialization.

3. Why Palette References Matter

Consider a show containing 75 cues that use the `Warm Amber` palette.

If those cues store literal RGB values, changing the show's definition of Warm Amber requires editing potentially 75 cues.

If those cues reference a first-class palette:

```
Warm Amber
UUID: 8F...
RGBW: ...
```

the user can modify the palette once.

Every cue intentionally referencing that palette can then resolve using the updated definition.

This is critical for large-show maintainability.

The same principle applies to:

- Colors
- Positions
- Beam configurations
- Gobos
- Intensity levels
- Other reusable fixture attribute combinations

4. Palette Resolution Belongs Below the UI

The UI may provide buttons, grids, favorites, swatches, etc., but these are presentations of the underlying model.

The UI should not own palette semantics.

Resolution should conceptually follow:

```
Cue
↓
Palette Reference
↓
Palette Resolver
↓
Fixture/Attribute Values
↓
Cue Engine
↓
Lighting Engine
```

The engine should ultimately receive deterministic fixture/attribute values.

This keeps palette behavior testable independently of SwiftUI.

5. Missing or Deleted Palette References

Define deterministic behavior for broken references.

If a cue references a palette that no longer exists, Aurora must never silently substitute unrelated values.

Recommended behavior:

- Detect the broken reference.
- Preserve enough information to diagnose it.
- Flag the cue/project through validation and UI diagnostics.
- Resolve to a documented safe behavior.
- Never crash the lighting engine.

Deleting a referenced palette should preferably warn the user before deletion.

Because Aurora has unlimited undo/redo, deletion and restoration of palettes must also correctly restore references.

6. Groups Should Also Be First-Class Entities

Fixture groups should follow the same philosophy.

A group should have:

- Stable UUID
- User-visible name
- Fixture membership
- Persistent project representation

Other objects should reference the group by ID rather than duplicating group membership.

Renaming a group must therefore not break cues, presets, MIDI mappings, songs, or other references.

7. Presets vs. Palettes

Avoid prematurely treating these as identical concepts merely because their initial implementation may look similar.

A palette generally represents reusable attribute information such as:

```
Warm Amber
Deep Blue
Center Stage
Drummer
Wide Fan
Open White
Gobo 3
```

A preset may represent a larger reusable programming construct or look.

For example:

```
Blue Stage Wash

Group: Back Wash
  Color → Deep Blue
  Intensity → 40%

Group: Front Wash
  Color → Open White
  Intensity → 65%

Group: Movers
  Position → Center Stage
```

The internal model should allow Aurora to evolve toward richer preset semantics without requiring destructive schema changes.

8. PR21: Song Mode Should Be an Orchestration Layer

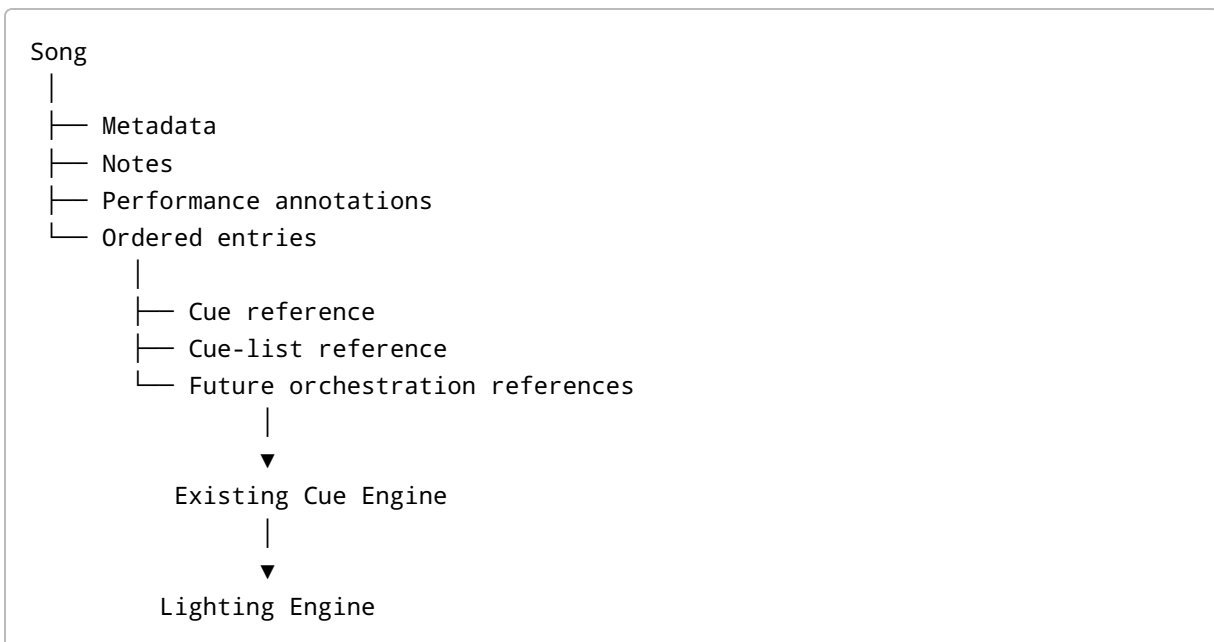
Song Mode should **not** become a second cue engine.

Aurora already has:

- Cue lists
- Cue resolution
- Playback
- Tracking
- Fades
- Effects
- Programmer
- Engine scheduling

Song Mode should orchestrate those systems rather than duplicate them.

Conceptually:

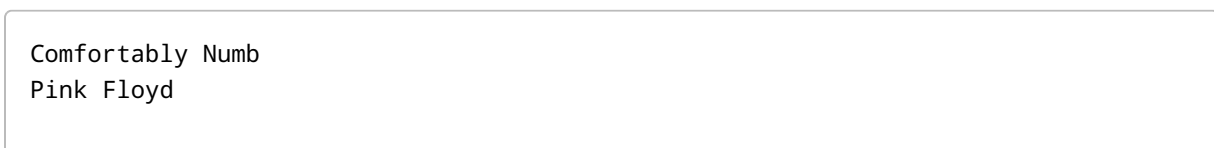


A Song describes **how the performance is organized**.

It should not independently calculate lighting output.

9. Example Song

Conceptually:



```
01  Intro
    → Cue 1.0

02  Verse 1
    → Cue 2.0

03  Chorus
    → Cue 3.0

04  Guitar Solo 1
    → Cue 4.0

05  Verse 2
    → Cue 5.0

06  Final Solo
    → Cue 6.0
```

The song entries reference existing cue/cue-list entities by stable ID.

If Cue 4.0 is edited, the Song does not need to be rewritten.

It still references the same entity.

10. Keep Song Metadata Separate From Lighting Behavior

Song Mode should own performance-oriented information such as:

- Title
- Artist
- Notes
- Set-list position
- Performer annotations
- Ordered cue/cue-list references
- Optional timing information
- Manual/automatic progression configuration

The existing playback/cue systems should continue owning:

- Fade calculations
- Tracking
- HTP/LTP
- Follow execution

- Cue resolution
- Programmer merging
- Effect evaluation
- Final DMX generation

Do not duplicate these responsibilities inside Song Mode.

11. Design for Future Song Capabilities Without Implementing Them Yet

The Song model should leave room for future capabilities such as:

- MIDI-triggered song navigation
- Set lists
- Song templates
- Tempo metadata
- Timecode
- Section markers
- Rehearsal notes
- Performer-specific annotations
- Remote/iPad navigation
- Song-level override mappings
- Automated progression
- Playback history

These features do not need to be implemented in PR21.

The requirement is simply:

Do not make today's Song model so rigid that these capabilities require replacing it later.

Avoid speculative complexity, but preserve clean extension points.

12. Stable Identity Is Critical

Use stable IDs throughout.

Preferred relationship:

```
SongEntry
  ↓ UUID reference
```


Cue / CueList

Not:

SongEntry

↓

"Cue 4"

User-visible names and cue numbers may change.

Identity should not.

This applies equally to:

- Fixtures
- Groups
- Palettes
- Presets
- Cue lists
- Cues
- Songs

13. Project Serialization

All new entities and references introduced in PR20/PR21 must work correctly with Aurora's existing document architecture.

Requirements:

- Codable/project serialization support
- Stable UUID persistence
- Schema migration consideration
- Round-trip tests
- Broken-reference validation
- Undo/redo correctness
- Project reopen correctness

A saved Aurora project must reconstruct the exact relationships between groups, palettes, presets, cues, and songs.

14. Testing Expectations

Add model and engine tests specifically covering reference semantics.

At minimum:

Palettes

- Create palette
- Rename palette without breaking references
- Modify palette and verify referencing cues resolve differently
- Literal cue values remain unchanged when palette changes
- Delete referenced palette
- Undo palette deletion
- Save/reload palette references
- Broken-reference handling

Groups

- Rename without breaking membership/reference
- Add/remove fixture membership
- Save/reload membership
- Undo/redo membership changes

Presets

- Apply preset
- Referenced palettes resolve correctly
- Save/reload preset relationships

Songs

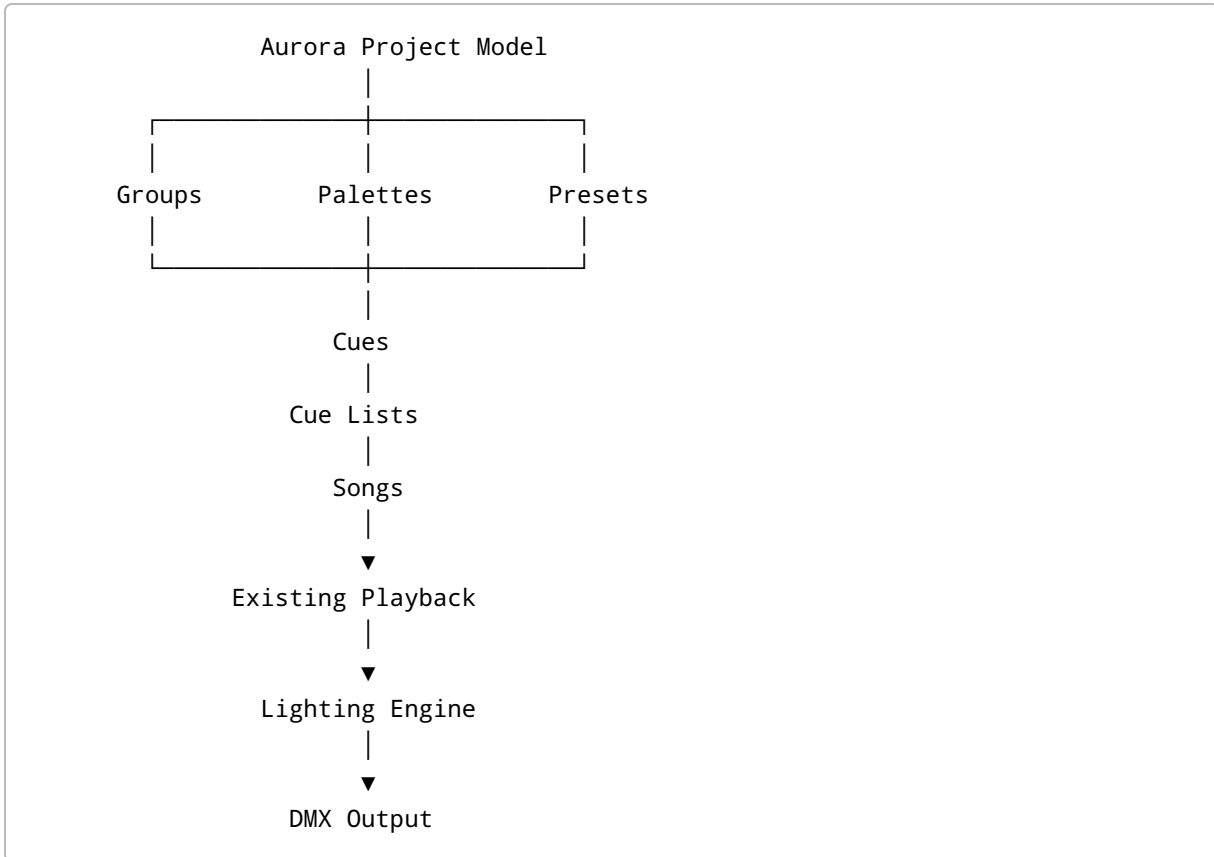
- Create song
- Add/reorder/remove entries
- Rename referenced cue without breaking song
- Edit referenced cue and verify song uses updated cue
- Save/reload song references
- Missing cue reference handling
- Manual progression
- Automatic progression where implemented

15. Architectural Rule

The guiding rule for PR20/PR21 is:

Reusable lighting concepts are model objects, not UI shortcuts. Songs orchestrate Aurora's existing playback system rather than replacing it.

The desired architecture is:



Maintain the existing Aurora separation between model, application core, engine, output, and UI.

Do not introduce UI-specific dependencies into the model or engine to implement these features.

Acceptance Principle

After PR20/PR21, Aurora should support this workflow:

1. User creates a color palette named `Warm Amber`.
2. User creates cues that intentionally reference `Warm Amber`.
3. Those cues are arranged into a song.
4. The user later changes `Warm Amber`.
5. Cues referencing that palette automatically use the new definition.
6. Literal colors stored in other cues remain unchanged.
7. The song requires no modification.

8. Cue playback continues through the existing deterministic cue/playback engine.
9. Saving and reopening the project preserves all relationships.

If that workflow is architecturally clean, deterministic, undoable, serializable, and independently testable, PR20/PR21 are heading in the right direction.